

1. 環境設定、テスト、保守及びエンドツーエンドチュートリアル

この文書は、リポジトリの取得からPython環境の確認、定義表の配置、回帰テスト、ラウンドトリップ成果物の生成及び一連のチュートリアルまでを一か所にまとめます。

変換方式の概念は [02_STRUCTURED_CSV_LHM_BINDINGS.md](#)、各プログラムの詳細はPhase別文書を参照してください。

Part A. 環境設定、定義、テスト及び保守

1. 目的

この文書は、UADC PoCで使用するGitHub作業環境の準備及び検証方法を説明します。編集可能な定義表及び生成済み定義表として、EN 16931、UBL、ADS及びISO 21378 ADCに関するファイルを示すとともに、必要なPython環境、テストプログラム及びコミット済みの出力証跡を説明します。

各処理の詳細は、次の文書に分けて記載しています。

- **01_ENVIRONMENT_TESTS_TUTORIAL.md**：短いエンドツーエンドの手順。
- **03_PHASE1_UBL_SYNTAX_BINDING.md**：Phase 1の仕様。
- **04_PHASE2_ADS_PSV_SEMANTIC_BINDING.md**：Phase 2の仕様。
- **02_STRUCTURED_CSV_LHM_BINDINGS.md**：共通モデル、タクソノミ及び支援ツールの仕様。

2. リポジトリの構成

ディレクトリ	役割
src/	Phase 1及びPhase 2の運用用変換プログラム。
src/tutorial/	運用用変換プログラムを呼び出す初心者向けラッパースクリプト。
tools/	定義の保守、タクソノミ生成、成果物生成及び確認用ツール。
tools/tutorial/	学習及び実験用の簡易変換プログラム。
tests/	直接実行できる回帰テスト及び支援プログラム。
specs/	LHM、結合定義、通貨及びXBRL GLの定義表。
samples/	コミット済みの入力XML及び期待するサンプルデータ。
out/	Gitで管理する、生成済みタクソノミ、Phase 1、Phase 2、逆変換、チュートリアル及び品質確認の証跡。
docs/	5種類の基準文書及びアーキテクチャ決定記録。

各スクリプトディレクトリには、GitHub上で参照するための短い **README.md** があります。docs/以下の5文書が、運用及び実装に関する基準文書です。

3. クローン及びPython環境の設定

3.1 Windows PowerShell

```
git clone https://github.com/pontsoleil/UADC-PoC.git
cd .\UADC-PoC
$python = 'python'
& $python --version
```

Pythonが **PATH** に登録されていない場合は、インストール済み実行ファイルのパスを指定します。

```
$python = 'C:\Users\  
<user>\AppData\Local\Programs\Python\Python310\python.exe'  
& $python --version
```

3.2 macOS又はLinux

```
git clone https://github.com/pontsoleil/UADC-PoC.git
cd ./UADC-PoC
PYTHON=python3
$PYTHON --version
```

3.3 主要プログラムのコンパイル確認

```
& $python -m py_compile `  
.\src\syntax_binding.py `  
.\src\semantic_binding.py `  
.\src\syntax_binding_ads_xbrl_gl.py
```

tools/ 以下のすべてのPythonプログラムは、次のコマンドで確認できます。

```
Get-ChildItem .\tools -Recurse -Filter *.py | ForEach-Object {  
    & $python -m py_compile $_.FullName  
}
```

4. 定義表

4.1 EN 16931 LHM

ファイル	用途
specs/lhm/source/EN16931_CIOUS_Invoice_LHM_Source.csv	ビジネス用語及び上書き列を含む編集用の元データ。
specs/lhm/EN16931_CIOUS_Invoice_LHM.csv	変換プログラム及びタクソノミ生成プログラムが使用する、生成済みの運用用LHM。
specs/lhm/EN16931_CIOUS_Invoice.xlsx	レビュー用のローカルワークブック。実行時の基準ではありません。

LHMには、セマンティック階層、クラス及び属性の型、多重度、実効的な **lhm_level**、セマンティックパス、要素名、データ型及びUBL XPathを記録します。

4.2 Phase 1 UBL構文結合

運用用UBL Invoice結合ファイルは、次のとおりです。

```
specs/bindings/syntax/EN16931_UBL_Invoice_Syntax_Binding.csv
```

このファイルには、クラス行とファクト行の両方が含まれます。XML上の位置、セクタ、構造化CSV列及び繰返し行スコープについて、実行時の基準となります。

4.3 Phase 2 ADS XBRL GL結合

specs/bindings/syntax/ 以下の6種類の構文結合CSVで、次の対象を定義します。

- Invoices Received
- Invoices Generated
- Invoices Received Lines
- Invoices Generated Lines
- Supplier Listing
- Customer Master

レビュー用ワークブックは、次のとおりです。

```
specs/bindings/ADS_XBRL_GL_Bindings.xlsx
```

4.4 Phase 2 ADS PSV結合

specs/bindings/semantic/ 以下の6種類のセマンティック結合CSVで、同じヘッダー、明細、仕入先及び得意先の対象ファミリを、区切り文字形式のファイルとして定義します。

4.5 ISO 21378 ADC結合

ISO 21378:2019の4種類のインボイスビューは、次のとおりです。

```
ISO21378_SAL_Invoice_Generated_CSV_Binding.csv
ISO21378_SAL_Invoice_Generated_Details_CSV_Binding.csv
ISO21378_PUR_Invoice_Received_CSV_Binding.csv
ISO21378_PUR_Invoice_Received_Details_CSV_Binding.csv
```

これらは、表38、表39、表53及び表54を表します。対応状況及び注記によって、直接対応、近似的対応、変換が必要な項目及びEN 16931インボイスから取得できないERP情報を区別します。

4.6 参照表

- **specs/Currency.csv** : ISO 4217通貨コードを補助通貨単位の桁数へ対応付けます。
- **specs/CountryCurrency.csv** : 国と通貨の例示データを提供します。
- **specs/XBRL-GL/** : 結合定義で使用するXBRL GL定義の参照ファイルを格納します。

5. サンプル及び生成済み証跡

5.1 Phase 1入力

```
samples/input/openpeppol_ubl_invoice_minimal.xml
samples/input/bis-billing3-examples/*.xml
```

現在の回帰テスト一式には、最小インボイス1件とBIS Billing 3の例9件が含まれます。

5.2 出力構成

```
out/taxonomy/
out/phase1/
out/phase2/ADS_XBRL_GL/
out/phase2/ADS_PSV/
out/phase2/ISO21378_ADC/
out/reverse/
out/tutorial/
```

レビュー用として、生成済み証跡をコミットしています。これらは手作業で編集せず、定義及びスキプトから再生成してください。

6. 実行環境の確認

チュートリアル的前提条件確認を実行します。

```
& $python .\src\tutorial\00_check_environment.py
```

このスクリプトは、3種類の運用用スクリプト、タクソノミ生成プログラム、LHM、UBL及びADS結合定義並びに最小サンプルを確認します。生成済みタクソノミがない場合は、元定義の不足ではなく、次に実行すべき処理として報告します。

必要に応じて、ローカルタクソノミを生成し、検証します。

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
& $python .\tests\validate_taxonomy.py
```

任意の依存パッケージ：

依存パッケージ 用途

Arelle	xBRL-CSVメタデータ、XBRLタクソノミ又はインスタンスの検証。
xmlschema	UBL 2.1ラウンドトリップXMLのスキーマ検証。
pypdf	PDFと照合したEN 16931対応範囲の監査。
pdfplumber	EN 16931表2からのLHM定義の更新。

7. 回帰テスト

テストは通常のPythonスクリプトであり、直接実行できます。

7.1 Phase 1

```
& $python .\tests\test_lhm_semantic_paths.py
& $python .\tests\test_lhm_hierarchical_csv_layout.py
& $python .\tests\test_syntax_binding.py
& $python .\tests\test_syntax_binding_reverse.py
& $python .\tests\test_bis_billing3_examples_conversion.py
& $python .\tests\test_roundtrip_artifacts.py
& $python .\tests\test_roundtrip_xml_ubl_schema.py
& $python .\tests\test_xbrl_csv_metadata_arelle.py
```

7.2 Phase 2 ADS XBRL GL

test_ads_*_xbri_gl.py という名前の対象別スクリプトをすべて実行した後、次のテストを実行します。

```
& $python .\tests\test_phase2_outputs_by_structured_csv_stem.py
```

7.3 Phase 2セマンティック出力

```
& $python .\tests\test_ads_invoices_received_psv.py
& $python .\tests\test_ads_invoices_generated_psv.py
& $python .\tests\test_semantic_binding_csv_format.py
& $python .\tests\test_semantic_binding_indexed_paths.py
```

7.4 全テストの直接実行

```
Get-ChildItem .\tests\test_*.py | Sort-Object Name | ForEach-Object {
    & $python $_.FullName
    if ($LASTEXITCODE -ne 0) { throw "Failed: $($_.Name)" }
}
```

直前に記録した実行では、22本のスクリプトがすべて成功しました。

8. テストの内部処理

テストでは、現在使用しているPython実行ファイルを指定して子プロセスのコマンドを作成します。そのため、変換プログラムとテストは同じ環境で実行されます。**phase1_helpers.py** は、対象別テストで使用するPhase 1結合パス及び変換支援処理を一元管理します。

ラウンドトリップ生成プログラムは、各テストケースについて、元XML、構造化CSV、メタデータJSON及び再生成UBL XMLの4種類の成果物ディレクトリを再作成します。スキーマテストは、バイト単位の一致ではなくXMLの妥当性を確認します。Arelleテストは、メタデータJSON及びそのタクソノミ参照を確認します。

対象別テストでは、セマンティックファクト、階層、セクタ、行スコープ、XBRL GLタプルの配置、金額の小数精度及び出力ファイル名を検証します。これらは単なるスモークテストではなく、回帰動作の仕様として機能します。

9. GitHubでの作業手順

作業開始前：

```
git status --short
```

```
git pull --ff-only
```

変更後：

1. 関係する個別テストを実行する。
2. 共通モデル又は変換プログラムを変更した場合は、回帰テスト一式を実行する。
3. コミット対象の出力証跡を再生成する。
4. 変更した文書のPDFを再生成する。
5. **git diff** 及び **git status** を確認する。
6. 目的のブランチへコミットし、プッシュする。

結合CSV及びLHMの元定義は、レビュー対象の元ファイルです。生成済みCSV、JSON、XML、XSD、リンクベース及びPDFの証跡は、これらの元ファイル及びコミット済みスクリプトから再現できなければなりません。

10. 文書PDFの作成手順

編集用の原本はMarkdownです。PDFは、VSCode Markdown PDF拡張機能を使用し、次の余白設定で生成します。

- 上下：20mm
- 左右：18mm

出力後は、最初と最後のページに加え、大きな表又はコードブロックを含むページを確認します。Markdownと対応するPDFは、同時にコミットしてください。

10.1 日本語文書の再生成

独自に作成したプロジェクト文書を含む各ディレクトリには、**ja/** サブディレクトリを設け、日本語Markdownを生成します。英語Markdownを構造上の原本とし、プロジェクト用語及び承認済みの日本語表現は、**docs/ja/TERMINOLOGY.csv** に定義します。

用語CSVは、次の列を使用します。

列	用途
source_term	元Markdownから検出する英語用語。
ja_term	生成文書へ挿入する日本語表現。
definition_ja	意図した意味を確認するための日本語定義。
match_mode	大文字・小文字を区別する識別子には literal 、自然言語の照合には phrase を使用。
notes	編集上の注意、略語又は複数形に関する注記。

CSVはExcelで編集し、**CSV UTF-8（コンマ区切り）** として保存します。保存後もUTF-8 BOMが維持されていることを確認してください。生成プログラムは **utf-8-sig** で読み込むため、BOMを受け

入れ、最初の列名からは除外します。

リポジトリのルートから、すべての日本語Markdownを再生成します。

```
$python =  
'C:\Users\nobuy\AppData\Local\Programs\Python\Python310\python.exe'  
& $python .\tools\translate_markdown_ja.py
```

ローカルのArgos Translate実行環境及び英日モデルは、事前に **.translation_runtime/** 及び **.translation_data/** 以下へ配置しておく必要があります。これらの大容量ローカル依存ファイルは、Gitの管理対象外です。生成プログラムは、文書本文を外部の翻訳サービスへ送信しません。

再生成せずに生成済み文書一式を検証する場合は、次のコマンドを実行します。

```
& $python .\tools\translate_markdown_ja.py --check
```

この検証では、対応対象の各英語Markdownに日本語版があること、コードフェンス数が一致すること及び各出力に日本語が含まれることを確認します。公開前には、用語、相対リンク、表、コマンド、パス、XPath及び未置換の保護マークがないことも確認してください。**ja_term** を修正すると、次回の全体生成時に、生成対象のすべての日本語文書へ反映されます。

11. トラブルシューティング

11.1 Pythonが見つからない

\$python 又は **PYTHON** に、実行ファイルの絶対パスを設定してください。

11.2 PDF関連ツールが依存パッケージを読み込めない

選択したPython環境へ必要なパッケージだけをインストールするか、任意のPDF由来の保守処理を省略してください。

11.3 Arelle又はUBLスキーマ検証を利用できない

それ以外のテストを実行し、省略した外部検証を明示的に報告してください。

11.4 生成結果が想定外に異なる

使用している結合ファイルのパス、入力ファイル名部分、文字エンコーディング、タクソノミの版、通貨表及び **tools/** の簡易チュートリアルではなく **src/** の運用用スクリプトで再生成したかどうかを確認してください。

12. 開発環境及び保守

開発及びツールガイド

この節では、クローン後の設定、ローカル開発時の確認及びモデル保守ツールを説明します。

ディレクトリの役割

src/	運用用変換スクリプト
tools/	セットアップ、モデル保守、タクソノミ生成及びテスト成果物の支援ツール
tests/	回帰テスト
specs/	コミット済みのLHM、結合定義、通貨及びモデル定義CSV
samples/	コミット済みのサンプルXML及び期待する出力
out/	Gitで管理する生成済みPoC証跡及び対象出力

インボイス変換の主要な実行プログラムは、**src/** にあります。セットアップ、再生成、テスト成果物の作成、タクソノミ出力及び仕様の保守には、**tools/** を使用します。

クローン後の初期設定

Windows PowerShell :

```
git clone https://github.com/pontsoleil/UADC-PoC.git
cd .\UADC-PoC
$python = 'python'
& $python --version
```

\$python = 'python' は、例で使用するPowerShell変数への代入です。**PATH** 上で利用できるPython実行ファイルを使用することを意味します。Pythonが **PATH** にはない場合は、実行ファイルの完全なパスを設定します。

```
$python = 'C:\Users\  
<user>\AppData\Local\Programs\Python\Python310\python.exe'  
& $python --version
```

macOS又はLinuxのシェル :

```
git clone https://github.com/pontsoleil/UADC-PoC.git
cd ./UADC-PoC
PYTHON=python3
$PYTHON --version
```

macOS又はLinuxでは **\$PYTHON script.py** を使用します。**& \$python script.py** は、Windows PowerShellだけで使用してください。

主要スクリプトのコンパイル

広範なテストを実行する前に、次のコンパイル確認を行います。

```
& $python -m py_compile `
.\src\syntax_binding.py `
.\src\syntax_binding_ads_xbrl_gl.py `
.\src\semantic_binding.py `
.\tools\build_roundtrip_test_artifacts.py `
.\tools\taxonomy\xBRLGL_TaxonomyGenerator.py
```

macOS／Linuxでの同等のコマンド：

```
$PYTHON -m py_compile \
./src/syntax_binding.py \
./src/syntax_binding_ads_xbrl_gl.py \
./src/semantic_binding.py \
./tools/build_roundtrip_test_artifacts.py \
./tools/taxonomy/xBRLGL_TaxonomyGenerator.py
```

ツールの分類

GitHub及び開発環境の支援：

```
build_roundtrip_test_artifacts.py
psv_viewer.html
```

変換対象モデル環境の支援：

```
taxonomy/xBRLGL_TaxonomyGenerator.py
build_lhm_from_source.py
build_syntax_binding.py
normalize_lhm_semantic_paths.py
normalize_lhm_class_element.py
check_lhm_class_element.py
audit_en16931_coverage.py
extend_en16931_lhm_coverage.py
order_lhm_by_en16931_table2.py
update_lhm_definitions_from_pdf.py
update_lhm_syntax_sequence_from_ubl_xsd.py
```

チュートリアル及びサンプル変換プログラム：

```
tools/tutorial/syntax_binding_sample.py
tools/tutorial/semantic_binding_sample.py
```

運用用変換プログラムは、**tools/** ではなく **src/** にあります。

モデル入力及び生成出力

コミット済みの元定義：

```
specs/lhm/EN16931_CIUS_Invoice_LHM.csv
specs/bindings/syntax/
specs/bindings/semantic/
specs/Currency.csv
```

ローカルで生成する出力：

```
out/taxonomy/
out/phase1/
out/phase2/
tests/roundtrip/
```

現在のPoCでは、GitHub上でPhase 1及びPhase 2の結果をレビューできるよう、生成済み証跡及び対象出力を含む **out/** をGitの管理対象としています。コミット済み出力は手作業で編集せず、元定義及びスクリプトから再生成してください。

tools/ 以下の15プログラムについて、目的、入出力、コマンドライン、処理、関数、検証、依存関係及び副作用の詳細な仕様を、**02_STRUCTURED_CSV_LHM_BINDINGS.md** に記載しています。

タクソノミの生成

主要スクリプト：

```
tools/taxonomy/xBRLGL_TaxonomyGenerator.py
```

役割：

```
LHM CSV -> 構造化CSVメタデータ用のローカルxBRL-CSVタクソノミ
```

回帰テストを使用して生成及び検証します。

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py  
& $python .\tests\validate_taxonomy.py
```

想定するローカル出力には、次のファイルが含まれます。

```
out/taxonomy/en16931/en16931-2026-07-05.xsd  
out/taxonomy/gen/gl-gen-2026-07-05.xsd  
out/taxonomy/plt/en16931-oim-2026-07-05.xsd  
out/taxonomy/plt/en16931-def-2026-07-05.xml
```

src/syntax_binding.py は、タクソノミファイルを生成しません。xBRL-CSVメタデータJSONを書き出す際に、**--taxonomy-base** を通じて生成済みファイルを参照します。

LHMの保守

スクリプト：

```
build_lhm_from_source.py  
normalize_lhm_semantic_paths.py  
normalize_lhm_class_element.py  
check_lhm_class_element.py  
audit_en16931_coverage.py  
extend_en16931_lhm_coverage.py  
order_lhm_by_en16931_table2.py  
update_lhm_definitions_from_pdf.py  
update_lhm_syntax_sequence_from_ubl_xsd.py
```

LHM編集後に通常実行する確認：

```
& $python .\tests\test_lhm_semantic_paths.py  
& $python .\tests\test_lhm_hierarchical_csv_layout.py  
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py  
& $python .\tests\validate_taxonomy.py
```

結合定義の保守

構文結合CSVは、次のディレクトリにあります。

```
specs/bindings/syntax/
```

運用用スクリプト :

```
src/syntax_binding.py  
src/syntax_binding_ads_xbrl_gl.py
```

構文結合の変更後 :

```
& $python .\tests\test_syntax_binding.py  
& $python .\tests\test_syntax_binding_reverse.py  
& $python .\tests\test_ads_supplier_listing_xbrl_gl.py  
& $python .\tests\test_phase2_outputs_by_structured_csv_stem.py
```

ADS XBRL GL結合を変更した場合は、対象別テストを実行し、代表的な生成インスタンスをArelleで検証してください。Supplier Listingを変更した場合は、**identifierType=V** セレクタと、その**identifierAddress** 子要素が、同じ仕入先識別子参照の中に維持されていることを確認します。

セマンティック結合CSVは、次のディレクトリにあります。

```
specs/bindings/semantic/
```

運用用スクリプト :

```
src/semantic_binding.py
```

セマンティック結合表は、対象表の定義を基礎として、**semantic_path**、**type** 及び **multiplicity** を追加します。変換プログラムは、**semantic_path** から元列を導出し、結合表の **type=C** 行から行スコープを導出します。

セマンティック結合の変更後 :

```
& $python .\tests\test_ads_invoices_received_psv.py  
& $python .\tests\test_ads_invoices_generated_psv.py  
& $python .\tests\test_semantic_binding_csv_format.py  
& $python .\tests\test_phase2_outputs_by_structured_csv_stem.py
```

ラウンドトリップ成果物の再生成

LHM、構文結合又はタクソノミメタデータの動作を変更した場合 :

```
& $python .\tools\build_roundtrip_test_artifacts.py
& $python .\tests\test_roundtrip_artifacts.py
```

成果物は、次の流れを示します。

```
元XML
構造化CSV
xBRL-CSVメタデータJSON
再生成UBL XML
```

開発継続時の作業手順

編集前：

```
git status --short
```

実行用スクリプトの編集後：

```
& $python -m py_compile .\src\syntax_binding.py
& $python .\tests\test_syntax_binding.py
& $python .\tests\test_syntax_binding_reverse.py
```

LHM又は結合CSVの編集後：

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
& $python .\tools\build_roundtrip_test_artifacts.py
& $python .\tests\test_roundtrip_artifacts.py
```

変更を共有する前に、広範なローカル確認を実行し、Arelle又はUBLスキーマキャッシュを利用できない場合は、その旨を明確に報告してください。

psv_viewer.html

tools/psv_viewer.html は、生成したADS PSVファイルをブラウザ上の表として表示します。すべてブラウザ内で動作し、ローカルWebサーバーを必要としません。パイプ、カンマ及びタブ区切り、行の絞り込み、ヘッダーの固定、横スクロール並びにすべてのデータ行が空欄である列の自動非表示に対応します。

13. 機械可読仕様ファイル

仕様ファイルガイド

この節では、**specs/** 以下の仕様ファイルと、その定義及び保守方法を説明します。

specs/ ディレクトリには、UADC PoCの変換処理で使用する機械可読CSV仕様を格納します。これらのファイルは、**src/** 及び **tools/** 以下のスクリプトへの入力です。ファイルは小さく、レビューしやすく、同じ入力から常に同じ結果を得られる状態に保ってください。

ディレクトリ構成

```
specs/  
  Currency.csv  
  CountryCurrency.csv  
  lhm/  
  bindings/  
  XBRL-GL/
```

通貨表

Currency.csv は、ISO 4217通貨コードを補助通貨単位の桁数へ対応付けます。XBRL GLの金額ファクトに **decimals** 値を設定するときに使用します。

代表的な列：

```
currency_code,minor_unit,currency_name
```

例：

```
JPY,0,Japanese Yen  
EUR,2,Euro
```

CountryCurrency.csv は、国の例を通貨コードへ対応付けます。フィンランド及びエストニアなど、国別の例示データとして使用します。

例：

```
FI,Finland,EUR,2,Euro  
EE,Estonia,EUR,2,Euro
```

実行時のXBRL単位には、引き続き **Currency.csv** のISO 4217通貨コードを使用します。

LHM仕様ファイル

LHMファイルは、次のディレクトリにあります。

```
specs/lhm/
```

主要ファイル：

- **EN16931_CIOUS_Invoice_LHM.csv**：構造化CSV変換及びタクソノミ生成で使用する、生成済みEN 16931インボイスLHM。
- **source/EN16931_CIOUS_Invoice_LHM_Source.csv**：生成済みLHMの再生成又は調整に使用する編集用CSV。
- **EN16931_CIOUS_Invoice.xlsx**：ローカルのレビュー用ワークブック。Gitの管理対象外であり、自動処理で更新しないでください。

LHMは、次の情報を定義します。

- セマンティックパス
- クラス名及び要素名
- 多重度
- 実効的な **lhm_level**
- UBL XPath結合への参照
- 構造化CSV及びxBRL-CSVタクソノミの生成に必要なフィールド

LHM元CSV

PDFから取得したEN 16931-1表2の個別項目を調整する場合は、**specs/lhm/source/EN16931_CIOUS_Invoice_LHM_Source.csv** を使用します。

生成手順：

```
python UADC_PoC/tools/build_lhm_from_source.py build `
  UADC_PoC/specs/lhm/source/EN16931_CIOUS_Invoice_LHM_Source.csv `
  UADC_PoC/specs/lhm/EN16931_CIOUS_Invoice_LHM.csv
```

任意の上書き列：

- **semantic_path_override**
- **class_term_override**
- **element_override**

element_override が空欄の場合、生成プログラムはセマンティックパスから一意のUpperCamelCase要素名を作成します。最後のパスセグメントが重複する場合は、一意となる最短のセマンティックパス接尾辞を使用します。

結合仕様ファイル

結合ファイルは、次のディレクトリにあります。

```
specs/bindings/
```

結合定義の作成及び変換規則は、次の文書に記載しています。

- 構文結合 : **03_PHASE1_UBL_SYNTAX_BINDING.md**
- セマンティック結合 : **04_PHASE2_ADS_PSV_SEMANTIC_BINDING.md**
- 共通の行スコープ及び関数単位の処理モデル : **02_STRUCTURED_CSV_LHM_BINDINGS.md**

結合ファイルは、次の関係を定義します。

- 元構文からUADCセマンティックモデルへの対応
- UADC構造化CSVから下流の対象構文への対応
- UADC構造化CSVからフラットなセマンティック対象表への対応

XBRL GL仕様ファイル

XBRL GL定義表は、次のディレクトリにあります。

```
specs/XBRL-GL/
```

主要ファイル：

- **xbrl-gl.csv** : **XBRL-GL-PWD-2016-12-01/gl/plt/case-c-b-m-u-e-t-s/gl-plt-all-2016-12-01.xsd** 及びimportされる **gl-cor**、**gl-bus**、**gl-muc**、**gl-ehm**、**gl-taf**、**gl-srcd**、**gl-usk** モジュールに整合するXBRL GL定義表。

この表は、利用可能な既存の英語及び日本語ラベルを保持します。順序、モジュール名、多重度、型名及び親子順序は、選択したXBRL GLタクソノミプロファイルから正規化します。

XPath 列には、**xbrli:xbrl** からのタプル又はファクトの絶対パスを記録します。このパスはタクソノミの親子ツリーから生成するため、結合表は内部行IDを保持せずに、対象XBRL GL上の位置を直接指定できます。

保守規則

- CSVファイルはUTF-8とし、表計算ソフトでの編集に必要な場合はUTF-8 BOMを使用する。
- 説明文にカンマが含まれることがあるため、引用符付きフィールドを保持できる構造化CSVライター又は表計算ソフトを使用する。
- レビュー用ワークブックが明示的な成果物でない限り、ローカルワークブックの変更をコミットしない。

- 生成プログラムがある場合は、生成済みファイルを手作業で編集せず、元CSV又はスクリプトから再生成する。
- 実行時に導出するデータは、それを結合仕様の一部として意図的に定義する場合を除き、結合表へ格納しない。

14. リポジトリのファイル及び証跡

リポジトリファイルガイド

この節では、リポジトリのサンプルファイル、期待値ファイル、参照資料及び図のファイルを説明します。

サンプル

コミット済みサンプルは、次のディレクトリにあります。

```
samples/
```

サブディレクトリ：

- **input/**：変換テストで使用する元XMLサンプル。
- **expected/**：個別の回帰テストで使用する安定した期待結果。

主要入力ファイル：

- **samples/input/openpeppol_ubl_invoice_minimal.xml**：OpenPeppol及びEN 16931の基準変換確認で使用する、最小UBL Invoiceサンプル。
- **samples/input/bis-billing3-examples/**：構文結合の広い対応範囲を確認するために選択したBIS Billing 3インボイス例。

BIS Billing 3の例では、値引き、VATカテゴリ、負の訂正インボイス、販売注文参照及び税務会計通貨の処理を確認します。

生成した構造化CSV、メタデータJSON及び再生成XMLは、**samples/**ではなく **tests/roundtrip/** 又は **out/** に格納します。

参照資料

軽量の参照資料は、次のディレクトリにあります。

```
references/
```

ファイル及びサブディレクトリ：

- **peppol_sources.md**：OpenPeppol及びBIS Billingの元資料に関する注記及びリンク。

- **figures/** : 提案資料から抽出又は作成したMarkdown文書用の画像。

確認済みの元ページ :

- Peppol BIS Billing 3.0 upcoming: <https://docs.peppol.eu/poacc/billing/3.0/upcoming/>
- UBL Invoice syntax tree: <https://docs.peppol.eu/poacc/billing/3.0/upcoming/syntax/ubl-invoice/tree/>
- EN 16931 model bound to UBL rules:
<https://docs.peppol.eu/poacc/billing/3.0/upcoming/rules/ubl-tc434/>

upcomingページでは、2026年5月リリースを示し、UBL Invoiceの構文文書と、UBLへ結合したEN 16931モデルの規則を分けて掲載しています。

基礎となるセマンティックモデルの作成に使用するローカルEN 16931資料には、**NEN-EN_16931-1_2017+A1_2019_en.pdf** が含まれます。6.2では、各情報要素及びグループの記述方法を定義しています。6.3の表2は、電子インボイスの中核要素に関するセマンティックデータモデルです。

現在のLHM拡張には、文書レベルの値引き、文書レベルの追加料金、文書合計、VAT内訳、インボイス明細、インボイス明細期間、インボイス明細値引き及びインボイス明細追加料金のEN 16931グループが含まれます。

OpenPeppol BIS Billingは、EN 16931セマンティックモデルの上に重ねるCIUS及び構文結合の元資料として扱い、LHMの一次資料とはしません。

図

図のファイルは、次のディレクトリにあります。

```
references/figures/
```

主要ファイル :

- **uadc_poc_processing_flow_figure1.png** : PoC提案書の図1。OpenPeppolインボイスXMLから階層型Tidy Dataを経て、下流の監査ビューへ至るUADCの2段階処理を示します。

図を追加する場合は、内容が分かる名前を付けた安定したPNGファイルを使用してください。追跡可能性のために役立つ場合に限り、中間レンダリングファイルも保持します。

15. 完全なテスト及びラウンドトリップ手順

テスト及びラウンドトリップ成果物ガイド

この節では、テストの実行及びラウンドトリップ成果物のレビュー方法を説明します。

コマンドは、**UADC_PoC** ディレクトリで実行します。テストスクリプトは通常のPythonスクリプトであり、**\$python** を使用して直接実行できます。pytestがインストールされている場合は、一部をpytestから実行することもできます。

支援ファイル

- **tests/roundtrip/** : レビュー可能なラウンドトリップ成果物。各ケースについて、元XML、構造化CSV、xBRL-CSVメタデータJSON及び再生成XMLを一緒に格納します。
- **tools/build_roundtrip_test_artifacts.py** : サンプルXML入力から、ラウンドトリップ成果物一式を再構築します。

Phase 1 構文結合テスト

次のテストは、UBL Invoice XMLから構造化CSVへの変換、xBRL-CSVメタデータ及びUBLラウンドトリップの動作を検証します。

```
& $python .\tests\test_openpeppol_invoice_conversion.py
& $python .\tests\test_lhm_hierarchical_csv_layout.py
& $python .\tests\test_syntax_binding.py
& $python .\tests\test_syntax_binding_reverse.py
& $python .\tests\test_ubl_schema_child_order.py
& $python .\tests\test_bis_billing3_examples_conversion.py
& $python .\tests\test_roundtrip_artifacts.py
```

Phase 1の順方向変換では、XMLの親コンテキストを再帰的にたどるため、入れ子の繰返しクラスでも、**dlInvoiceLine** などの親ディメンション値を、**dlItemAttributes** などの子ディメンションとともに保持します。

逆変換では、**--ubl-schema-root** 又は **--ubl-schema-url** を使用して、XSDファイルからUBL子要素順を導出できます。**test_ubl_schema_child_order.py** は、UBLパッケージ全体をダウンロードせずに、このXSD由来の順序付け処理を確認します。

test_syntax_binding_reverse.py は、スコープをまたぐ絶対XPathの処理も確認します。BT-90はルート of **AccountingSupplierParty** の下へ書き込み、**PaymentMeans/Invoice** は作成してはなりません。**test_bis_billing3_examples_conversion.py** は、**Allowance-example.xml** の通貨で絞り込まれた合計を確認し、BT-110が **1225.00**、BT-111が **9324.00** になることを検証します。

Phase 2 ADS XBRL GLテスト

次のテストは、最初にPhase 1構造化CSVを再生成し、その後、**specs/bindings/syntax/** 以下のADS XBRL GL構文結合CSVを使用します。

```
& $python .\tests\test_ads_invoices_received_xbrl_gl.py
& $python .\tests\test_ads_invoices_generated_xbrl_gl.py
& $python .\tests\test_ads_invoices_received_lines_xbrl_gl.py
& $python .\tests\test_ads_invoices_generated_lines_xbrl_gl.py
& $python .\tests\test_ads_supplier_listing_xbrl_gl.py
& $python .\tests\test_ads_customer_master_xbrl_gl.py
```

想定出力：

```
out/phase2/ADS_XBRL_GL/<structured-csv-stem>/<target-view>.xbrl
```

対象ビューファイル：

```
Invoices_Received.xbrl
Invoices_Generated.xbrl
Invoices_Received_Lines.xbrl
Invoices_Generated_Lines.xbrl
Supplier_Listing.xbrl
Customer_Master.xbrl
```

test_ads_supplier_listing_xbrl_gl.py は、売手の名称及び識別子が **identifierType=V** の識別子参照へ書き込まれ、売手の番地、市区町村、国及び郵便番号が、その **gl-bus:identifierAddress** タブルの下へ書き込まれることを確認します。

Phase 2 ADS PSV及びCSVテスト

次のテストは、**specs/bindings/semantic/** 以下のセマンティック結合ファイルを使用します。

```
& $python .\tests\test_ads_invoices_received_psv.py
& $python .\tests\test_ads_invoices_generated_psv.py
& $python .\tests\test_semantic_binding_indexed_paths.py
& $python .\tests\test_semantic_binding_csv_format.py
& $python .\tests\test_phase2_outputs_by_structured_csv_stem.py
```

想定出力：

```
out/phase2/ADS_PSV/<structured-csv-stem>/<target-view>.psv
```

タクソノミ及びLHMの確認

```
& $python .\tests\test_lhm_semantic_paths.py
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
& $python .\tests\validate_taxonomy.py
```

ラウンドトリップ成果物

ラウンドトリップの流れ：

```
元XML -> 構造化CSV -> 再生成XML
```

成果物は、次のディレクトリにあります。

```
tests/roundtrip/<dataset>/
  source_xml/
  structured_csv/
  metadata_json/
  roundtrip_xml/
```

現在のデータセット：

```
tests/roundtrip/openpeppol-minimal/
tests/roundtrip/bis-billing3-examples/
```

再構築：

```
$python = 'python'
& $python .\tools\build_roundtrip_test_artifacts.py
```

検証：

```
& $python .\tests\test_roundtrip_artifacts.py
```

このテストは、元XML、構造化CSV、メタデータJSON及びラウンドトリップXMLの対応を確認します。また、**InvoiceNumber**、**DocumentCurrencyCode**、**InvoiceLineIdentifier**、金額の**currencyID**、タクソノミ・エントリーポイント及びxBRL-CSV列と概念の対応など、代表的な値を確認します。

任意の検証

Arelleによるメタデータ検証：

```
& $python .\tests\test_xbrl_csv_metadata_arelle.py
```

UBL 2.1スキーマ検証：

```
& $python .\tests\test_roundtrip_xml_ubl_schema.py
```

再生成XMLは、元XMLとバイト単位で同一になることを想定していません。結合済みCSV値から再構築するため、XML宣言の形式、名前空間宣言の位置、インデント及び結合対象外のXML内容が異なる場合があります。

現在のテスト実行報告

直近の記録日：

2026-07-14

記録結果：

PASS

2種類の簡易変換実装を **tools/tutorial/** へ移動した後、2026年7月14日に **tests/test_*.py** の回帰テスト22本すべてを実行し、すべて成功しました。**tools/** 以下のPythonプログラム14本も、すべて **py_compile** に成功しました。

対象範囲には、EN 16931 LHM駆動の構文結合変換、構造化CSV生成、xBRL-CSVメタデータ生成、Arelle検証、UBL逆変換、BIS Billing 3サンプル変換、LHM確認及びローカルタクソノミ生成確認が含まれます。

タクソノミ及びLHM確認、OpenPeppol変換、BIS Billing 3の9件すべての変換、構造化CSVメタデータ、10件のラウンドトリップ成果物及び10件すべてのxBRL-CSVメタデータに対するArelle検証が成功しました。絶対パスによる通貨絞り込みXPathの評価を修正し、**Allowance-example.xml** (はBT-110を **1225.00**、BT-111を **9324.00** として出力するようになりました。すべてのPhase 2 ADS XBRL GL及びADS PSV/CSV出力を再生成し、それぞれのテストが成功しました。

その後、Supplier Listing XBRL GL結合へ、**identifierType=V** の識別子参照の下に売手の郵便住所ファクトを追加しました。Phase 1入力10件すべてからSupplier Listingを再生成し、生成した10件の **Supplier_Listing.xbrl** インスタンスは、すべてArelle検証に成功しました。4種類のISO 21378 ADCインボイス結合もPhase 1入力10件すべてに適用し、40件の対象CSVファイルを生成しました。ISOのフィールド別対応範囲及び既知の元データ不足は、

04_PHASE2_ADS_PSV_SEMANTIC_BINDING.md 第19章に記録しています。

これらの結果により、計画したPhase 1及びPhase 2 PoCの基準範囲が完成しました。ここでいうISO 21378部分の完成とは、計画した4種類のインボイスビュー、その対応関係、出力及び不足分類が完成したことを意味し、EN 16931にISO 21378で定義されたすべての監査システム項目が含まれることを意味しません。

逆変換プログラムは、セマンティックな子項目が繰返し構文コンテキストの外側に格納される場合でも、絶対結合XPathをUBL文書ルート基点のまま処理します。これにより、BT-90が **PaymentMeans** の下へ入れ子の **Invoice** を生成することを防ぎます。**xmlschema** を利用できる環境でテストした場合、再生成した10件のラウンドトリップXMLは、すべてUBL 2.1 Invoiceスキーマ検証に成功します。

Part B. エンドツーエンドチュートリアル

1. 目的

このチュートリアルでは、長いコマンドを組み立てることなく、UADC の基本的な処理の流れを確認します。**src/tutorial/** 以下のスクリプトは、Phase 1 及び Phase 2 の本番処理で使用するものと同じ変換プログラムを呼び出します。

すべてのコマンドは、リポジトリのルートディレクトリで実行してください。

2. チュートリアルの流れ

```
UBL Invoice XML
-> Phase 1 構文結合
-> 構造化CSV及び xBRL-CSV メタデータ
-> 構文結合による逆変換
-> 再生成された UBL Invoice XML

構造化CSV
-> Phase 2 ADS 構文結合
-> ADS Invoices Received XBRL GL
```

チュートリアルの出力は、**out/tutorial/** 以下に生成されます。

3. 実行環境の確認

Windows PowerShell :

```
$python = 'python'
& $python .\src\tutorial\00_check_environment.py
```

このスクリプトは、次の内容を報告します。

- 使用する Python 実行ファイル
- 検出したリポジトリのルートディレクトリ
- 不足している実行用スクリプト又は定義ファイル
- 生成済みの OIM タクソノミ・エントリーポイントが存在するかどうか

内部では、**REQUIRED_PATHS** に、LHM、UBL及びADSの結合定義、サンプルXML、実行用変換プログラム、ラウンドトリップ生成プログラム並びにタクソノミ生成プログラムが列挙されていま

す。必要な元ファイルが一つでも不足している場合、スクリプトは終了ステータス **1** を返します。

4. Phase 1 構造化CSVの生成

```
& $python .\src\tutorial\01_convert_sample_to_structured_csv.py
```

入力：

```
samples/input/openpeppol_ubl_invoice_minimal.xml
```

出力：

```
out/tutorial/openpeppol_ubl_invoice_minimal.csv  
out/tutorial/openpeppol_ubl_invoice_minimal.json
```

ラッパースクリプトは、最初に **ensure_taxonomy** を呼び出します。ローカルにタクソノミが存在しない場合は、タクソノミ生成の回帰テスト用スクリプトを実行します。その後、EN 16931 UBL 構文結合定義、メタデータの出力パス及びタクソノミの基点を指定して、**src/syntax_binding.py** を呼び出します。

CSVを開き、次の点を確認してください。

- インボイス行に **dlInvoice** が設定されていること
- 繰り返されるインボイス明細又はVAT内訳が、それぞれ別の行に出力されていること
- 各クラスが所有しない列のセルが空欄になっていること
- LHMの要素定義からファクト列名が設定されていること

JSONを開き、OIMの文書型、タクソノミ・エントリーポイント、テーブルテンプレート、ディメンション、概念及び通貨単位を確認してください。

5. UBL XMLへのラウンドトリップ

```
& $python .\src\tutorial\02_roundtrip_structured_csv_to_xml.py
```

出力：

```
out/tutorial/openpeppol_ubl_invoice_minimal.roundtrip.xml
```

必要な構造化CSVがまだ生成されていない場合、**ensure_structured_csv** が前節のチュートリアル処理を実行します。ラッパースクリプトは、同じUBL構文結合定義と **--reverse** オプションを指定

して、**src/syntax_binding.py** を呼び出します。

再生成されるXMLは、意味的に等価な出力であり、元ファイルとバイト単位で同一になることを目的としていません。名前空間宣言の位置、インデント、XML宣言及び結合対象外の内容は異なる場合があります。重要な確認点は、結合された値、階層、UBL子要素の順序及びスキーマ適合性です。

6. Phase 2 ADS XBRL GLビューの生成

```
& $python .\src\tutorial\03_generate_ads_xbrl_gl.py
```

出力ディレクトリ：

```
out/tutorial/xbrl-gl/
```

ラッパースクリプトは、次のファイルを使用します。

```
src/syntax_binding_ads_xbrl_gl.py  
specs/bindings/syntax/ADS_Invoices_Received_XBRL_GL_Binding.csv  
specs/lhm/EN16931_CIOUS_Invoice_LHM.csv  
specs/Currency.csv
```

生成されるインスタンスには、XBRLコンテキスト及び単位に加え、対象ビューに必要なXBRL GLのタプル階層が含まれます。

7. 生成結果の確認

7.1 構造化CSV

引用符で囲まれたCSVセルを正しく保持できる表計算ソフトウェアを使用してください。親のインボイス行と、繰り返される子行に、異なるクラスが所有するファクトが混在していないことを確認します。

7.2 メタデータJSON

タクソノミ・エントリーポイントが次の値になっていることを確認します。

```
out/taxonomy/plt/en16931-oim-2026-07-05.xsd
```

7.3 ラウンドトリップXML

次のコマンドを実行します。

```
& $python .\tests\test_syntax_binding_reverse.py  
& $python .\tests\test_roundtrip_xml_ubl_schema.py
```

7.4 XBRL GL

生成したインスタンスをArelleで開きます。従来のタプル指向サンプルと同じ表示を期待するのではなく、タプル階層及びファクトが正しく生成されていることを確認してください。

7.5 PSV又はCSV

区切り文字形式のPhase 2出力を確認する場合は、**tools/psv_viewer.html** を使用します。このビューアはローカルファイルを読み込み、パイプ、カンマ及びタブの各区切り文字に対応します。また、行の絞り込み、ヘッダーの固定表示及び全セルが空欄の列の非表示が可能です。

8. 内部で行われる処理

チュートリアルの上ラッパースクリプトは、**subprocess.run** を **check=True** で呼び出します。そのため、子プロセスである変換プログラムが失敗した場合、ラッパースクリプトも直ちに停止します。

Phase 1では、内部で次の処理を行います。

1. 結合定義のクラス行及びファクト行を読み込む。
2. ディメンション及びクラスが直接所有するフィールドを導出する。
3. XMLのクラスコンテキストを再帰的にたどる。
4. 親行及び繰り返される子行を出力する。
5. CSVと同じ列及びディメンション構成を使用してメタデータを書き出す。

逆変換では、ディメンションによって行をグループ化し、結合定義の絶対XPathからXMLノードを再構築し、述語及び属性を適用した後、スキーマから導出した構文順序に従ってUBL子要素を並べ替えます。

ADS XBRL GLの生成では、ファクトの所有関係を検証し、対象ビューに必要な元データ行を選択し、コンテキスト及び通貨単位を作成します。その後、対象XPathをたどり、XBRL GLスキーマの順序に従ってファクトを書き出します。

9. 学習用の簡易実装

tools/tutorial/ 以下のプログラムは、前節までで使用したラッパースクリプトとは異なります。これらは、学習及び結合定義の実験を目的とした小規模な変換プログラムです。

```
tools/tutorial/syntax_binding_sample.py  
tools/tutorial/semantic_binding_sample.py
```

これらの簡易実装は、完全な階層型の行所有規則、繰返しスコープ、メタデータ、UBLへの逆変換又はPhase 2のすべての処理を実装していません。PoCの成果物を生成するためではなく、小規模なアルゴリズムを理解するために使用してください。

10. 次のステップ

チュートリアルを完了した後は、次の文書を参照してください。

1. LHM、ディメンション及び構造化CSVの規則については、
02_STRUCTURED_CSV_LHM_BINDINGS.md を参照する。
2. Phase 1の結合定義を変更する前に、**03_PHASE1_UBL_SYNTAX_BINDING.md** を参照する。
3. ADS PSV、ADS XBRL GL又はADCを生成する前に、
04_PHASE2_ADS_PSV_SEMANTIC_BINDING.md を参照する。
4. 関連する回帰テスト及び外部検証を実行する場合は、
01_ENVIRONMENT_TESTS_TUTORIAL.md を参照する。